Lab 1 — HCL Components & First AWS Deployment

Learning Objectives

By the end of this lab you will be able to:

1. Create a Terraform project with the correct file structure and `required_providers` configuration.
2. Configure the AWS provider and authenticate to an AWS account.
3. Use a data source to look up information that already exists in AWS.
4. Declare a resource and provision an EC2 instance.
5. Run the full Terraform workflow: `init` → `fmt` → `validate` → `plan` → `apply` → `destroy`.
6. Apply a lifecycle meta-argument and observe how it changes Terraform's behaviour.

Part 1 — Create the Project (20 min)

Step 1.1 — Create the directory structure

```
mkdir -p ~/ipa26/lab01
cd ~/ipa26/lab01
```

Create three empty files. Terraform loads every `.tf` file in the directory, so splitting by purpose is a convention for humans, not a requirement of the tool.

```
touch terraform.tf provider.tf main.tf
```

File	Purpose
`terraform.tf`	The `terraform` settings block (version constraints)
`provider.tf`	Provider configuration
`main.tf`	Resources and data sources

Step 1.2 — Write the terraform settings block

Put the following in `terraform.tf`:

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

Understand what you just wrote:

- `required_version` constrains the Terraform CLI itself.
- `required_providers` tells Terraform which provider to download and from where.
- `source = "hashicorp/aws"` is a registry address — shorthand for `registry.terraform.io/hashicorp/aws`.
- `~> 5.0` is a pessimistic constraint: allow 5.x, but not 6.0.

> **Question 1:** Why is pinning the provider version important for a team project? Write your answer in your lab report.

Step 1.3 — Configure the AWS provider

Put the following in `provider.tf`:

```
provider "aws" {
  region = "us-east-1"

  default_tags {
    tags = {
      Course    = "IPA-Terraform"
      ManagedBy = "Terraform"
    }
  }
}
```

`default_tags` applies these tags to every taggable resource created by this provider — a good habit for tracking who created what.

Step 1.4 — Initialize

```
terraform init
```

Expected result: Terraform downloads the AWS provider and creates a `.terraform/` directory and a `.terraform.lock.hcl` file.

> **Checkpoint 1:** Run `ls -a`. Confirm you see `.terraform/` and `.terraform.lock.hcl`. Open `.terraform.lock.hcl` and note the exact provider version that was selected.

Part 2 — Add a Data Source (20 min)

A data source reads information that Terraform does not manage. Here,we ask AWS for the most recent Amazon Linux 2023 AMI, instead of hard-coding an AMI ID that will go stale in next 3 months.

Step 2.1 — Look up the AMI

Add to `main.tf`:

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
```

```
  filter {
    name   = "name"
    values = ["al2023-ami-2023.*-x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}
```

Note the block structure here:

- `data` is the **block type**
- `"aws_ami"` and `"amazon_linux"` are **labels** (type and name)
- `most_recent` and `owners` are **arguments**
- `filter { ... }` is a **subblock**, which may be repeated

Step 2.2 — Also look up the default VPC

```
data "aws_vpc" "default" {
  default = true
}
```

Step 2.3 — Inspect the data source values

Run a plan and then use the console to explore.

```
terraform plan
terraform console
```

Inside the console, type:

```
> data.aws_ami.amazon_linux.id
> data.aws_ami.amazon_linux.name
> data.aws_vpc.default.id
```

Press `Ctrl+D` to exit.

> **Checkpoint 2:** Record the AMI ID and the default VPC ID in your lab report.

Part 3 — Create Your First Resource (30 min)

Step 3.1 — Add a security group

Add to `main.tf`:

```
resource "aws_security_group" "web" {
  name        = "lab01-web-sg"
  description = "Allow HTTP inbound and all outbound"
  vpc_id      = data.aws_vpc.default.id
```

```
  ingress {
    description = "HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "lab01-web-sg"
  }
}
```

Step 3.2 — Add the EC2 instance

```
resource "aws_instance" "web" {
  ami                    = data.aws_ami.amazon_linux.id
  instance_type          = "t3.micro"
  vpc_security_group_ids = [aws_security_group.web.id]

  user_data = <<-EOF
    #!/bin/bash
    dnf install -y nginx
    echo "<h1>Hello from Terraform - $(hostname)</h1>" > /usr/share/nginx/html/ind
ex.html
    systemctl enable --now nginx
  EOF

  tags = {
    Name = "lab01-web-server"
  }
}
```

Step 3.3 — Format, validate, plan

```
terraform fmt
terraform validate
terraform plan
```

Read the plan output carefully before continuing. Answer these in your report:

> **Question 2:** How many resources will be added?
>
> **Question 3:** Which attributes show as `(known after apply)`, and why can Terraform not know them yet?

Step 3.4 — Apply

```
terraform apply
```

Type `yes` when prompted.

Step 3.5 — Verify

```
terraform show
aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=lab01-web-server" \
  --query "Reservations[].Instances[].[InstanceId,PublicIpAddress,State.Name]" \
  --output table
```

Open the public IP in a browser. You should see your "Hello from Terraform" page. (Allow 1–2 minutes for nginx to install.)

> **Checkpoint 3:** Take a screenshot of the working web page and include it in your report.

Part 4 — Meta-Arguments and Lifecycle (25 min)

Step 4.1 — Observe a destructive change

Change the instance type in `main.tf`:

```
instance_type = "t3.small"
```

Run:

```
terraform plan
```

> **Question 4:** Does Terraform plan to update in place or to replace the instance? Look for the `~` or `-/+` symbol and explain what it means.

Revert the change back to `t3.micro` before continuing.

Step 4.2 — Add create_before_destroy

Add a `lifecycle` block to `aws_instance.web`:

```
resource "aws_instance" "web" {
  # ... existing arguments ...

  lifecycle {
    create_before_destroy = true
  }
}
```

Change the instance type to `t3.small` again and run `terraform plan`.

> **Question 5:** How did the plan output change compared to Step 4.1? Why would this matter for a production web server?

Revert to `t3.micro`.

Step 4.3 — Try prevent_destroy

Replace the `lifecycle` block with:

```
lifecycle {
  prevent_destroy = true
}
```

Now run:

```
terraform apply
terraform destroy
```

> **Question 6:** What error do you get, and at what stage does it occur — plan or apply?

Remove the `prevent_destroy` line when you have recorded the answer.

Part 5 — Clean Up (5 min)

This step is mandatory.

```
terraform destroy
```

Confirm with `yes`, then verify nothing is left:

```
aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=lab01-web-server" \
            "Name=instance-state-name,Values=running,pending" \
  --query "Reservations[].Instances[].InstanceId" --output text
```

An empty result means you are done.

Deliverables

- `terraform.tf`, `provider.tf`, `main.tf` — your final working configuration
- `lab01-report.md` containing:
  - Answers to Questions 1–6